Write-Behind and Write-Around

absorb huge write rates, or keep writes out of the cache

Watch the 4-min video

In one line: write-behind (write-back) acknowledges writes in the cache and flushes them to the database later in batches, turning floods of tiny writes into a few big ones at the risk of losing unflushed data; write-around sends writes straight to the database and skips the cache, so write-heavy, rarely read data doesn't push hot data out.

1 Fifty thousand tiny writes a second

Every video view increments a counter in your database. A viral video gets 50,000 views a second, and the database melts under 50,000 tiny `UPDATE`s on one row (each one locks that row, so they queue). Does every single view need its own database write right now? Not really.

2 Write-behind (write-back)

The application writes to the **cache** and gets an immediate acknowledgement. The cache, or a background job, **flushes** changes to the database later, often in batches. Writes become very fast, and many updates to the same key are **combined** into one database write: a counter incremented 500,000 times becomes one `UPDATE … SET views = views + 500000`.

WORKED EXAMPLE: VIDEO VIEW COUNTERS

Each view runs `INCR views:{video_id}` in Redis, which takes microseconds. Every 10 seconds a worker reads the counters (atomically, with `GETDEL` or a rename), writes the totals to the database in one batch, and starts from zero. 50,000 writes per second become a few thousand batched updates every 10 seconds, one per active video.

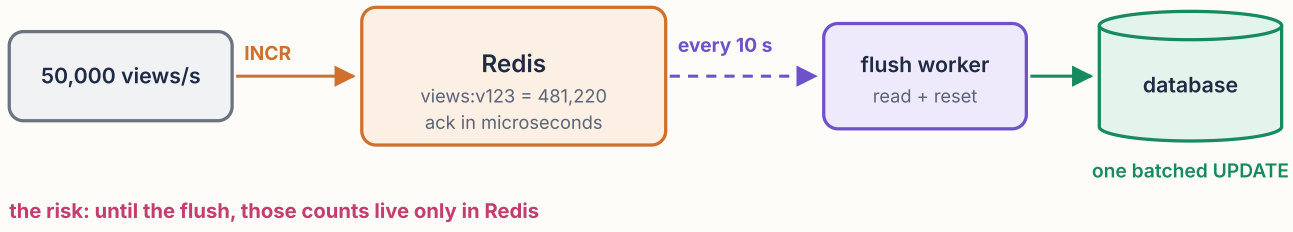


Figure 1. Write-behind: Redis absorbs the increments; a worker flushes the totals to the database every 10 seconds.

3 The risk: durability

Until the flush, the data lives **only in the cache**. If Redis crashes, the last 10 seconds of counts are gone. Redis persistence (AOF with `fsync` every second) and replication shrink that window, but don't close it. And anyone reading the database sees slightly stale values until the next flush.

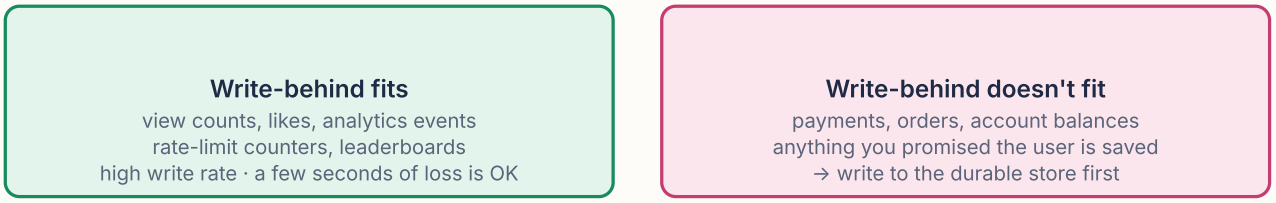


Figure 2. Use write-behind only where losing a few seconds of updates is acceptable.

4 Write-around

Write-around is the opposite. Writes go **straight to the database**, skipping the cache entirely, and any existing cache entry for that key is invalidated. The cache only gets the data later, *if* someone reads it, through cache-aside or read-through.

It suits data that's written often but read rarely, or not soon: logs, audit records, bulk imports, archived content. With write-through, those writes would flood the cache and **evict hot data**. Write-around keeps the cache full of what's actually read. The trade-off: the first read after a write is always a miss.

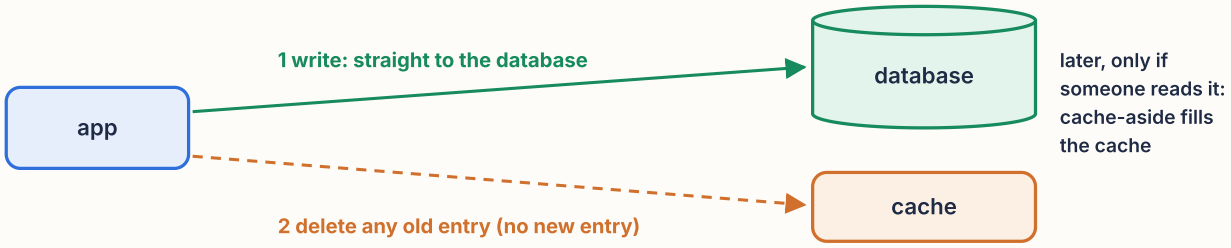


Figure 3. Write-around: the write goes only to the database; the cache gets the item later, and only if it's read.

5 The four write patterns side by side

Pattern	Write goes to	Write latency	Risk	Good for
Cache-aside (delete)	DB, then delete key	DB only	small stale window	most read-heavy data
Write-through	cache → DB, synchronous	higher (both)	caching unread data	read-soon-after-write
Write-behind	cache, DB later	lowest	losing unflushed writes	counters, analytics
Write-around	DB only	DB only	first read is a miss	logs, imports, audit

COMMON MISTAKES

- Write-behind for orders or payments, then losing them in a cache failover.
- A flush job that reads then resets in two steps, losing increments that arrive in between: use an atomic swap.
- No monitoring of flush lag, so a stuck worker silently stops data reaching the database.

6 Key terms

Write-behind / write-back: ack in cache, flush to DB later

Coalescing writes: many updates to one key become one write

Durability window: how much acknowledged data could be lost

Flush: writing buffered changes to the database

Write-around: writes skip the cache

Cache pollution: rarely read data evicting hot data

7 Interview questions

Q1. When is write-behind acceptable? BASIC

When the write volume is high and losing a few seconds of updates is acceptable, like view counters, likes and analytics, since data sits in the cache until the asynchronous flush.

Q2. What is write-around good for? BASIC

Write-heavy, read-rarely data like logs, audit records and bulk imports, because writes go only to the database and the cache isn't polluted with entries nobody reads.

Q3. Design a like counter for posts with millions of likes per minute. SCENARIO

Increment per-post counters in Redis (sharded if one key is too hot), flush totals to the database every few seconds in batches, serve counts from Redis, and accept that a crash can lose a few seconds of likes.

Q4. A nightly job imports 10 million records. How do you keep it from wrecking the cache? SCENARIO

Write-around: insert straight into the database without touching the cache (only invalidate keys that exist), so the import doesn't evict the hot working set.

Q5. How do you make write-behind safer? TRICKY

Persist the cache (AOF every second) and replicate it, flush frequently, use atomic read-and-reset, monitor flush lag, or put the writes on a durable log like Kafka and aggregate from there.

Q6. Why do 50,000 single-row updates per second hurt the database even if each is fast? TRICKY

They all lock the same row, so they serialise; each also writes the WAL and indexes. Batching collapses them into one write per interval.

Next: SD-F-005 · TTL and Expiration backendroadmap.com